

实验四：序列检测器

实验日期：2025.11.14
姓名： 何宗霖
学号：23374275
班级：231516
任课教师：李家军

- 一、实验目的
- 1. 掌握序列检测器的原理，及 Verilog 实现。
 - 2. 掌握有限状态机的写法。
 - 3. 培养分析、设计和调试数字电路的能力。

二、实验原理

【序列检测器】

序列检测器用于识别特定的输入序列并产生相应的输出信号。通过状态机（如有限状态机）来监控输入信号的变化。检测器根据当前输入状态和历史输入状态进行状态转移，只有当输入序列与预设的目标序列完全匹配时，输出信号才会被激活。

在非重叠序列检测中，设计确保了在检测到一个有效序列后，系统会进入一个特殊状态以防止下一序列的重叠，从而提高了序列检测的准确性和可靠性。例如序列 111011011011011101,待检测序列 1101,检测结果:111011011011011101。

【Moore 型状态机（检测 1101）】

Moore 状态机的输出仅依赖于当前状态而与输入无关。想要输出 out = 1，状态 S3 必须形成。Mealy 状态机的输出与当前状态和输入有关。想要输出 out = 1，状态 S2 下输入 din = 1 就可以了，不需要状态 S3。根据波形可对比反应更快。

本次实验采用 Moore 型状态机，一共分为五种状态：

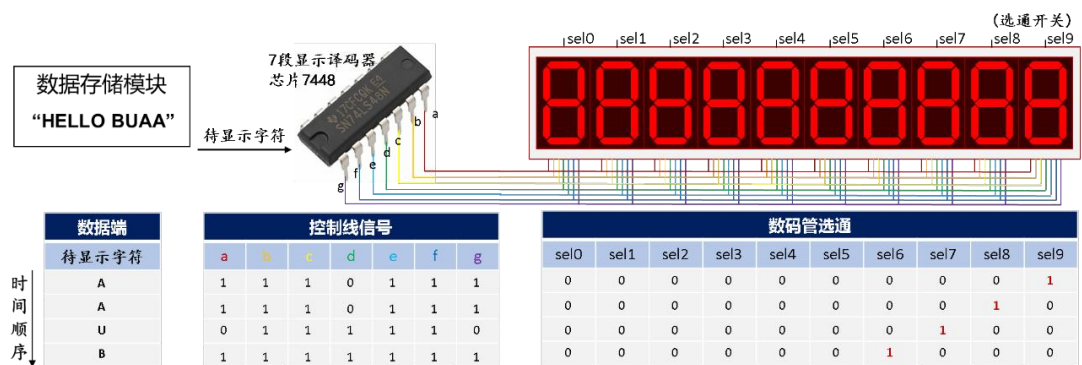
- S0: 初始状态 (包含序列为 XX00, X010 共 6 种情况)
- S1: 检测到第一位 1 (包含序列为 X001, 0101 共 3 种情况)
- S2: 检测到第二位 1 (包含序列为 XX11 共 4 种情况)
- S3: 检测到第三位 0 (包含序列为 X110 共 2 种情况)
- S4: 检测到第四位 1，序列 1101 检测完成 (包含序列为 1101 共 1 种情况)

➤ 状态转移

	S0	S1	S2	S3	S4
输入1	S1	S2	S2	S4	S1
输入0	S0	S0	S3	S0	S0

【动态扫描原理：分时复用控制线】

为解决控制线过多的问题，采用多个像素共用同一根控制线；为使得每个数码管显示内容一样，则增加选通开关，每一时刻只有一个数码管显示。



为解决没有同时显示全部内容，利用视觉暂留效应，即光对视网膜所产生的视觉在光停止作用后,仍保留一段时间的现象，如果在视觉暂留效应消失前，数码管再一次被选通，即可实现连续显示效果。

引入扫描速度（刷新率）指标，即每秒数码管被选通的次数；引入扫描方式即同时点亮的行数与整个区域行数的比例。

三、实验内容

- 1.（必做）重叠检测
- 2.（必做）非重叠检测

实现可配置待检测序列的非重叠 4 位序列检测器

- 配置待检测序列：通过 4 个拨码开关来设置待检测序列
- 输入：使用 2 个点动开关来输入序列的下一位，分别代表输入 1 和 0
- 显示：右边四个数码管应显示出当前的 4 位序列，当检测成功时左边数码管显示“yes”
- 可重置：清零端口被触发时输入序列清空。

四、硬件资源

设备仪器：FPGA 芯片（EP4CE10F17C8N）

【外设：点动开关】

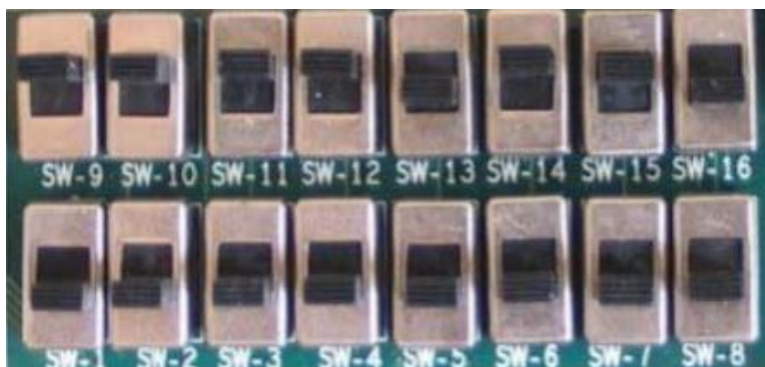


KSI 拨下时 F7-F10 可用

电动开关未按下时输出高电平，按下时输出低电平。

F1			CON1. 10	AB14
F2			CON1. 11	W20
F3			CON1. 12	AA15
F4			CON1. 13	AB15
F5			CON1. 14	T15
F6			CON1. 15	U20
F7	SMBUS_SDA	由开关 KSI 选择	CON1. 16	AA18
F8	SMBUS_SCL	由开关 KSI 选择	CON1. 17	AA16
F9	I2C_SCL	由开关 KSI 选择	CON1. 18	AB18
F10	I2C_SDA	由开关 KSI 选择	CON1. 19	AB19

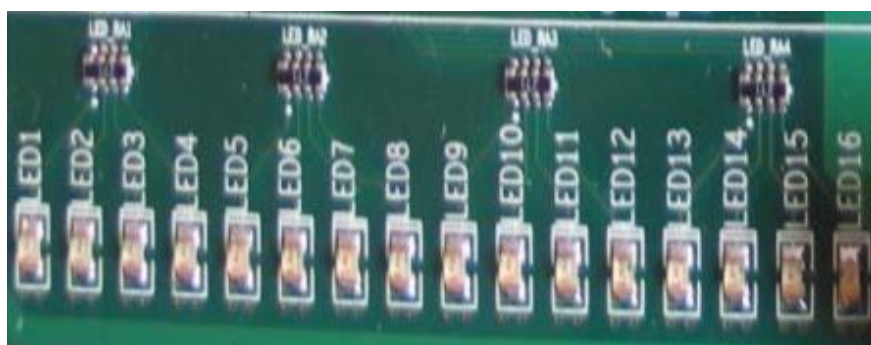
【外设：拨码开关】



开发平台上有二组 16 只拨码开关输入，开关拨置上为逻辑“1”电平，开关拨置下为逻辑“0”电平。

第一组：SW1，SW2，SW3，SW4，SW5，SW6，SW7，SW8 已经固定连接到实验平台中的 FPGA_CON1 和 FPGA_CON2 处；第二组：SW9，SW10，SW11，SW12，SW13，SW14，SW15，SW16 如果使用该模块，请把控制拨码开关模块 CTRL_SW 中开关 SEL1，SEL2 拨置于下逻辑电平为 00，使 DP9 数码管显示 1。

【外设：LED 指示灯】



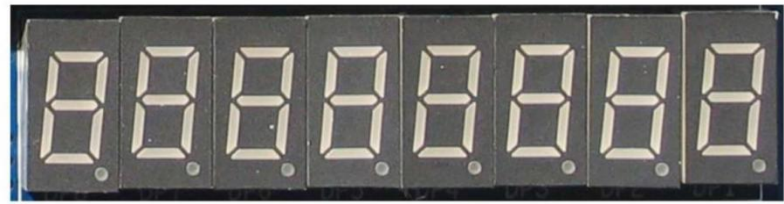
开发平台上有 2×8 共 16 个 LED 指示灯，便于对用户的操作给出提示，使用户得到相应的反馈信息，也可验证设计的正确性，低电平时亮。

第一组：LED1，LED2，LED3，LED4，LED5，LED6，LED7，LED8。如果使用该模块，请把控制拨码开关模块 LCD_ALONE_CTRL_SW 中开关 VLPO 拨置于下为低电平来确定。

第二组：LED9，LED10，LED11，LED12，LED13，LED14，LED15，LED16。如果使用该模块，请把控制拨码开关 CTRL1_SW 中 SEL1 拨置于上，SEL2 拨

置于下，逻辑电平为 10，使 DP9 数码管显示 2。

【外设：数码管】



seg控制一个数码管哪一段是否发光

设计端口	引脚分配	开发平台模块
Seg[7]	U21	8xSEG LA
Seg[6]	R22	8xSEG LB
Seg[5]	T17	8xSEG LC
Seg[4]	R21	8xSEG LD
Seg[3]	P22	8xSEG LE
Seg[2]	R17	8xSEG LF
Seg[1]	R20	8xSEG LG
Seg[0] (高使能)	P20	8xSEG LH
Sel[1]	AB17	8xSEG DS2
Sel[0] (低使能)	T18	8xSEG DS1

数码管段选

Sel为数码管使能端

控制某一个数码管是否被使能

sel[7]	B12	W21	8xSEG DS8
sel[6]	K20	AA20	8xSEG DS7
sel[5]	J18	AA19	8xSEG DS6
sel[4]	H11	AB20	8xSEG DS5
sel[3]	L7	AA17	8xSEG DS4
sel[2]	L8	AB16	8xSEG DS3
sel[1]	K9	AB17	8xSEG DS2
sel[0]	H6	T18	8xSEG DS1

数码管位选

五、实验过程与结果

1. 重叠检测。

实验核心代码：（用的是一段式）

```
module SequenceDetector #(
    parameter delayT = 250_000 // e.g. 对于 50MHz 时钟约 5ms 延迟
)()
    input wire clk_sys,
    input wire resetn_b,
    input wire [3:0] sw, // pattern 输入
    input wire key1_b, // 输入“1”
    input wire key0_b, // 输入“0”
    output wire [7:0] seg, //
    output wire [7:0] sel

    // 去抖输入
    wire resetn, key1, key0;
    debouncing#(delayT(delayT)) db_rst (.clk_sys(clk_sys), .key_b(resetn_b), .key(), .key_pulse(resetn));
    debouncing#(delayT(delayT)) db_k1 (.clk_sys(clk_sys), .key_b(key1_b), .key(), .key_pulse(key1));
    debouncing#(delayT(delayT)) db_k0 (.clk_sys(clk_sys), .key_b(key0_b), .key(), .key_pulse(key0));
    //用去抖模块进行实例化

    // 核心检测逻辑(一段式)
    reg [3:0] input_shift; // 用户输入的最近4位
    reg [3:0] pattern;
    reg match; // 匹配结果

    always @(posedge clk_sys or negedge resetn) begin
        if (!resetn) begin
            input_shift <= 4'b0000;
            pattern <= 4'b0000;
            match <= 1'b0;
        end else begin
            pattern <= sw; //sw是可以选的需要匹配的模式
            if (!key1) begin // 检测到输入1
                input_shift <= {input_shift[2:0], 1'b1};
            end else if (!key0) begin // 检测到输入0
                input_shift <= {input_shift[2:0], 1'b0};
            end
            //else if 而不是写两个if 是为了防止两个同时按下
            // 更新匹配状态
            match <= (input_shift == pattern);
            //这里是有两个时钟序列的延迟的，就是输入新的一位时就已经match了，但是input_shift需要再等一个时钟周期才能变，再过一个时钟周期match才变
        end
    end
end
```

(!key0) 这里写 else if 而不是写两个 if 是为了防止两个同时按下。

match 这里是有两个时钟序列的延迟的，就是输入新的一位时就已经 match 了，但是 input_shift 需要再等一个时钟周期才能变，再过一个时钟周期 match 才变


```

wire [7:0] seg_data [7:0]; // 8个8位的数组，前三位输出7段显示数码管显示，第4位空，第5-8位输出结果

// “YES” / “NO” 前三个数码管显示
assign seg_data[7] = match ? 8'h76 : 8'hEC; // Y / N 7+6 01110110 (小写的y)14+12 11101100 (小写的n)
assign seg_data[6] = match ? 8'h9E : 8'hFC; // E / O 15+12 11111100
assign seg_data[5] = match ? 8'hB6 : 8'h00; // S / space

// 中间空一位
assign seg_data[4] = 8'h00;

// 后4位显示输入序列
assign seg_data[3] = input_shift[3] ? 8'h60 : 8'hFC; // 1 / 0
assign seg_data[2] = input_shift[2] ? 8'h60 : 8'hFC;
assign seg_data[1] = input_shift[1] ? 8'h60 : 8'hFC;
assign seg_data[0] = input_shift[0] ? 8'h60 : 8'hFC;

// 合成为 64 位输入—就是前面的8个8位的，ScanDisplayDriver读的时候8位8位的读
// 因为一个seg7段码就是需要8位，相当于每一个展示都要读8位
wire [63:0] display_data = {
    seg_data[7], seg_data[6], seg_data[5], seg_data[4],
    seg_data[3], seg_data[2], seg_data[1], seg_data[0]
};

// 实例化显示驱动模块
ScanDisplayDriver #(
    .CLOCK_FREQ(25_000_000),
    .REFRESH_HZ(1000)
) disp_driver (
    .clk_sys(clk_sys),
    .resetn(resetn),
    .display_data(display_data),
    .seg(seg),
    .sel(sel)
);

```

endmodule

显示部分：用了 8 个 8 位的 seg_data 用于存储 8 个数码管上应该显示的内容，并用 display_data 将这 64 位合并，便于传入驱动模块进行实例化。

防抖模块：

```

module debouncing_fsm #(
    parameter bitwidth = 20,
    parameter delayT = 250_000 // e.g. 对于 50MHz 时钟约 5ms 延迟 50*10^6
)(
    input wire clk_sys, // 系统时钟
    input wire resetn, // 异步复位（低有效）
    input wire key_b, // 原始按键信号（低有效）
    output reg key, // 去抖后的按键信号（低有效，保持按下时间）
    output reg key_pulse, // 仅一个时钟周期的脉冲（低有效）
    output reg [bitwidth-1:0] counter
);

//=====
// 1. 状态定义（有状态的都要记录，counter这种++也算，记录状态的都是reg）
//=====
localparam IDLE = 2'b00; // 等待按下
localparam COUNTING = 2'b01; // 去抖计数中
localparam PRESSED = 2'b10; // 按下稳定

reg [1:0] current_state, next_state; // 三个状态所以两位
// reg [bitwidth-1:0] counter; // 移到输出了

//=====
// 2. 状态寄存器（不赋具体的值，描述下一个时钟信号来了之后做什么）（初始化需要赋具体的值）
//=====
always @(posedge clk_sys or negedge resetn) begin
    if (!resetn) begin
        current_state <= IDLE;
        counter <= delayT; // 因为是倒计时
    end else begin
        current_state <= next_state;
        counter <= counter; // 合并了next_counter
    end
end
end

```

顺序：状态定义->状态寄存器->状态转移->输出和计数

1.状态定义：一定要全

2.状态寄存器：一定包含两部分，必须用时序逻辑

(1) resetn 下降沿置零

常见代码：if (!resetn) begin
 current_state 置零
 End

(2) 时钟信号上升沿：current_state=next_state, counter=next_counter

```
// 3. 状态转移逻辑 （组合逻辑）
//=====
always @(*) begin
    next_state = current_state; // 默认保持
    case (current_state)
        IDLE: begin
            counter<=delayT; //赋值，必要的，不同于初始化
            if (key_b == 1'b0) // 检测到按下
                next_state = COUNTING;
        end
        COUNTING: begin
            if (key_b == 1'b1) begin // 中途松开（或者是一开始跳变防抖的核心！！）
                counter<=delayT; //
                next_state = IDLE;
            end else if (counter > 0) begin
                // 消抖计数中
                counter <=counter -1;
                next_state <=COUNTING;
            end else begin //counter==0,计数结束，按键稳定
                next_state = PRESSED;
                counter <=0;
            end
        end
        PRESSED: begin
            counter <=0;
            if (key_b == 1'b1) // 松开
                next_state = IDLE;
        end
    endcase
end
```

1.状态转移：一定包含 next_state=current_state/current_state 修改 + 必须用组合逻辑

(1)默认状态不变

next_state=current_state

(2)状态改变

如果有输入就是 if (! 输入)begin
 end else begin
 end

如果没有输入只有状态就是 case (current_state):
 current_stateA: begin
 end
 current_stateB: begin
 End

```

// 4. 状态输出与计数逻辑（输出一定是阻塞赋值）
//=====
reg prev_key;

always @(posedge clk_sys or negedge resetn) begin
    if (!resetn) begin
        key    <= 1'b1;
        prev_key <= 1'b1;
    end else begin
        case (current_state)
            IDLE: begin
                key <= 1'b1; // 未按下
            end
            COUNTING: begin
                key <= 1'b0; // 稳定按下
            end
            PRESSED: begin
                key <= 1'b0; // 稳定按下
            end
        endcase

        // 生成单周期脉冲：下降沿有效（低电平）
        prev_key <= key;
    end
end

//仅当key下降沿，key_pulse为0
assign key_pulse=~(prev_key&(~key));

```

2.输出和计数：输出尽量用 assign 的组合逻辑驱动模块：

```

module ScanDisplayDriver #(
    parameter integer CLOCK_FREQ = 25_000_000, // 默认 25MHz
    parameter integer REFRESH_HZ = 1000        // 每个数码管刷新频率 (Hz)
)()
    input          clk_sys,    // 系统时钟
    input          resetn,    // 低有效复位
    input [63:0]    display_data, // 每个数码管的段码输入（高有效）
    output reg [7:0] seg,      // 段选输出 {a,b,c,d,e,f,g,dp} 高有效
    output [7:0]    sel        // 位选输出（低有效）
);

// -----
// 参数 / 计数器，用于刷新和 1s 滚动定时
// -----
localparam REFRESH_COUNTER_WIDTH = 16; // 决定扫描速率
reg [REFRESH_COUNTER_WIDTH-1:0] refresh_cnt;

// 使用 refresh_cnt 的高 3 bit 做index索引（固定0..7循环）
wire [2:0] index = refresh_cnt[REFRESH_COUNTER_WIDTH-1 -: 3];

// 当前扫描的数码管索引（0~7）
reg [2:0] digit_idx;
always @(posedge clk_sys or negedge resetn) begin
    if (!resetn) begin
        refresh_cnt <= 0;
        digit_idx <= 0;
    end else begin
        // refresh_cnt始终+1，提供稳定的index 0--7循环
        refresh_cnt <= refresh_cnt+1;

        //选通数码管根据index切换
        digit_idx <= index;
    end
end

// 输出驱动逻辑
always @(*) begin
    seg = display_data[digit_idx*8+7 -:8]; // 输出当前选中的段码
    //每次都取display_data的8位
    //如果写成 digit_idx*8+7 +: 8，则是 [digit_idx*8 : digit_idx*8 + 7]
end

// sel 低有效：只有当前位为 0，其余为 1
assign sel = ~(8'b0000_0001 << digit_idx); //选通的位置会变，不是只显示一位，是8个数码管都会显示

endmodule

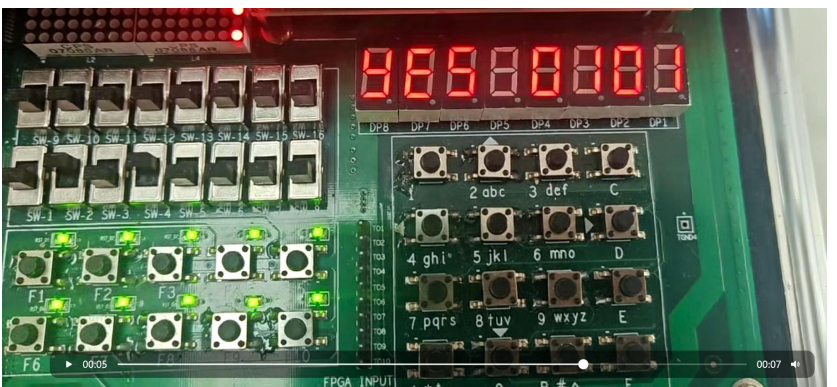
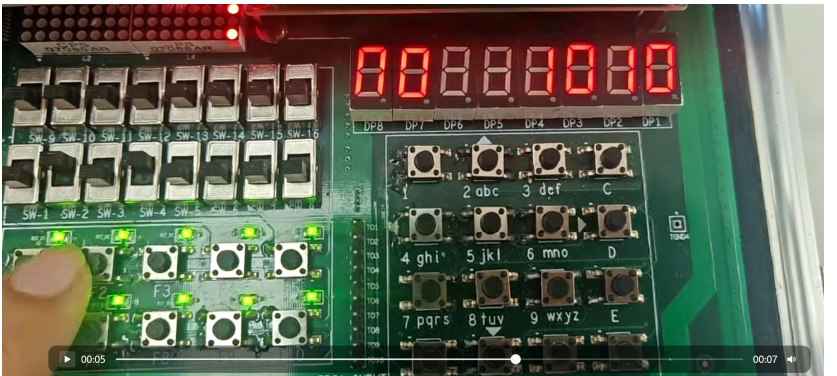
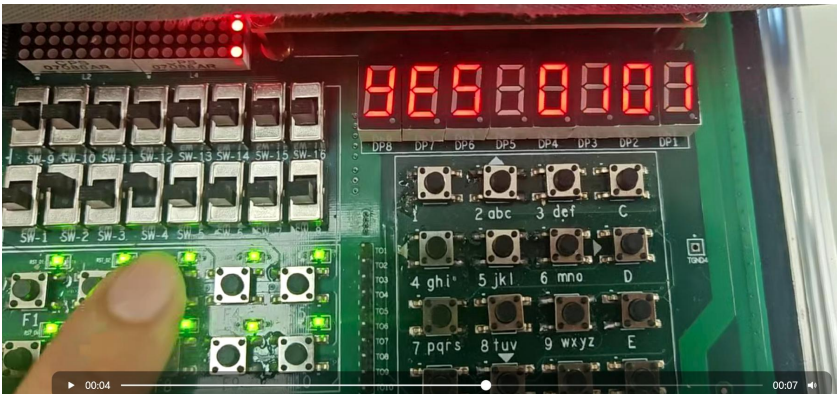
```


主要作用：利用 index，控制 seg 背后的值，以及选通的数码管（每次只选通一位）。通过高频移动选通数码管，实现视线滞留。

管脚绑定情况界面：

Node Name	Direction	Location	I/O Bank	VREF Group	Pin Location	I/O Standard	Reserved	Current Strength	Slew Rate	Differential Pair
clk_sys	Input	PIN_T1	2	B2_N0	PIN_T1	2.5 V ...fault		8mA (...ault)		
key0_b	Input	PIN_AA15	4	B4_N2	PIN_AA15	2.5 V ...fault		8mA (...ault)		
key1_b	Input	PIN_W20	5	B5_N2	PIN_W20	2.5 V ...fault		8mA (...ault)		
resetn_b	Input	PIN_AB14	4	B4_N2	PIN_AB14	2.5 V ...fault		8mA (...ault)		
seq[7]	Output	PIN_U21	5	B5_N1	PIN_U21	2.5 V ...fault		8mA (...ault)	2 (default)	
seq[6]	Output	PIN_R22	5	B5_N1	PIN_R22	2.5 V ...fault		8mA (...ault)	2 (default)	
seq[5]	Output	PIN_T17	5	B5_N2	PIN_T17	2.5 V ...fault		8mA (...ault)	2 (default)	
seq[4]	Output	PIN_R21	5	B5_N1	PIN_R21	2.5 V ...fault		8mA (...ault)	2 (default)	
seq[3]	Output	PIN_P22	5	B5_N1	PIN_P22	2.5 V ...fault		8mA (...ault)	2 (default)	
seq[2]	Output	PIN_R17	5	B5_N2	PIN_R17	2.5 V ...fault		8mA (...ault)	2 (default)	
seq[1]	Output	PIN_R20	5	B5_N1	PIN_R20	2.5 V ...fault		8mA (...ault)	2 (default)	
seq[0]	Output	PIN_P20	5	B5_N1	PIN_P20	2.5 V ...fault		8mA (...ault)	2 (default)	
sel[7]	Output	PIN_W21	5	B5_N2	PIN_W21	2.5 V ...fault		8mA (...ault)	2 (default)	
sel[6]	Output	PIN_AA20	4	B4_N1	PIN_AA20	2.5 V ...fault		8mA (...ault)	2 (default)	
sel[5]	Output	PIN_AA19	4	B4_N1	PIN_AA19	2.5 V ...fault		8mA (...ault)	2 (default)	
sel[4]	Output	PIN_AB20	4	B4_N1	PIN_AB20	2.5 V ...fault		8mA (...ault)	2 (default)	
sel[3]	Output	PIN_AA17	4	B4_N1	PIN_AA17	2.5 V ...fault		8mA (...ault)	2 (default)	
sel[2]	Output	PIN_AB16	4	B4_N2	PIN_AB16	2.5 V ...fault		8mA (...ault)	2 (default)	
sel[1]	Output	PIN_AB17	4	B4_N1	PIN_AB17	2.5 V ...fault		8mA (...ault)	2 (default)	
sel[0]	Output	PIN_T18	5	B5_N2	PIN_T18	2.5 V ...fault		8mA (...ault)	2 (default)	
sw[3]	Input	PIN_M16	5	B5_N2	PIN_M16	2.5 V ...fault		8mA (...ault)		
sw[2]	Input	PIN_R19	5	B5_N1	PIN_R19	2.5 V ...fault		8mA (...ault)		
sw[1]	Input	PIN_AA14	4	B4_N2	PIN_AA14	2.5 V ...fault		8mA (...ault)		
sw[0]	Input	PIN_AA1	2	B2_N1	PIN_AA1	2.5 V ...fault		8mA (...ault)		
<< next node >>										

最终结果：



3.非重叠检测。

实验核心代码：（用的是三段式）

```
module SequenceDetector_NonOverlap #(
    parameter delayT = 250_000 // e.g. 对于 50MHz 时钟约 5ms 延迟
)(
    input wire clk_sys,
    input wire resetn_b,
    input wire [3:0] sw, // pattern 输入
    input wire key1_b, // 输入“1”
    input wire key0_b, // 输入“0”
    output wire [7:0] seg,
    output wire [7:0] sel
);

// -----
// 去抖输入
// -----
wire resetn, key1, key0;
debouncing#(delayT) db_rst (.clk_sys(clk_sys), .key_b(resetn_b), .key(), .key_pulse(resetn));
debouncing#(delayT) db_k1 (.clk_sys(clk_sys), .key_b(key1_b), .key(), .key_pulse(key1));
debouncing#(delayT) db_k0 (.clk_sys(clk_sys), .key_b(key0_b), .key(), .key_pulse(key0));
// 啥意思

// 三段式
// 状态转换
reg [3:0] current_state; // 是已有的状态
reg [3:0] next_state; // 是 即将 到来的状态，应该是现在的状态等于下一个状态
reg [3:0] pattern;
reg [2:0] len; // 表示1-4,4个数之内的输入无效
reg [2:0] next_len;
always @(posedge clk_sys or negedge resetn) begin
    if (!resetn) begin
        current_state <= 4'b0000; // 不太对吧,对的，就是要让现在的下一个状态清零
        // next_state <= 4'b0000 相当于还要再等一个时钟周期
        pattern <= 4'b0000;
        len <= 3'b0;
    end else begin
        pattern <= sw; // sw是可以选的需要匹配的模式
        current_state <= next_state;
        len <= next_len;
    end
end

end
```

```

//组合逻辑
always @(*) begin
    //默认状态保持
    next_state = current_state;
    next_len = len;

    if (!key1) begin // 检测到输入1
        next_state = {current_state[2:0], 1'b1}; //不能放到里面，无论如何输入都要改变
        if(match)begin
            next_len=1;
        end
        else if(len < 3'd4)begin
            // next_state = {current_state[2:0], 1'b1};
            next_len = len+1;
        end
        else begin
            next_len = 3'd4;
        end
    end
    else if (!key0) begin // 检测到输入0
        next_state = {current_state[2:0], 1'b0}; //不能放到里面，无论如何输入都要改变
        if(match)begin
            next_len = 1;
        end
        else if(len < 3'd4)begin
            // next_state = {current_state[2:0], 1'b0}; //不能放到里面，无论如何输入都要改变
            next_len = len+1;
        end
        else begin
            next_len = 3'd4;
        end
    end
end
//输出逻辑
// always @(*) begin
//     match = (current_state == pattern)&& (len==3'd4);
// end

assign match = (current_state == pattern)&& (len==3'd4); //不会一直是1，条件不成立自然为0

// 显示逻辑：8位数码管
// 显示格式：
// [7:5] YES / NO
// [3:0] 用户输入位（1 显示 “1”，0 显示 “0”）

wire [7:0] seg_data [7:0]; //8个8位的数组，前三位输出7段显示数码管显示，第4位空，第5-8位输出结果

// “YES” / “NO” 前三个数码管显示
assign seg_data[7] = match ? 8'h76 : 8'hEC; // Y / N 7+6 0110110 (小写的y)14+12 11101100 (小写的n)
assign seg_data[6] = match ? 8'h9E : 8'hFC; // E / O 15+12 11111100
assign seg_data[5] = match ? 8'hB6 : 8'h00; // S / space

// 中间空一位
assign seg_data[4] = 8'h00;

// 后4位显示输入序列
//这里是current还是next? ?? 如果是current的话会过一个时钟周期才变
assign seg_data[3] = next_state[3] ? 8'h60 : 8'hFC; // 1 / 0
assign seg_data[2] = next_state[2] ? 8'h60 : 8'hFC;
assign seg_data[1] = next_state[1] ? 8'h60 : 8'hFC;
assign seg_data[0] = next_state[0] ? 8'h60 : 8'hFC;

// 合成为 64 位输入——就是前面的8个8位的，ScanDisplayDriver读的时候8位8位的读
// 因为一个seg7段码就是需要8位，相当于每一个展示都要读8位
wire [63:0] display_data = {
    seg_data[7], seg_data[6], seg_data[5], seg_data[4],
    seg_data[3], seg_data[2], seg_data[1], seg_data[0]
};

// -----
// 实例化显示驱动模块
// -----
ScanDisplayDriver #(
    .CLOCK_FREQ(25_000_000),
    .REFRESH_HZ(1000)
) disp_driver (
    .clk_sys(clk_sys),
    .resets(resetn),
    .display_data(display_data),
    .seg(seg),
    .sel(sel)
);
endmodule

```

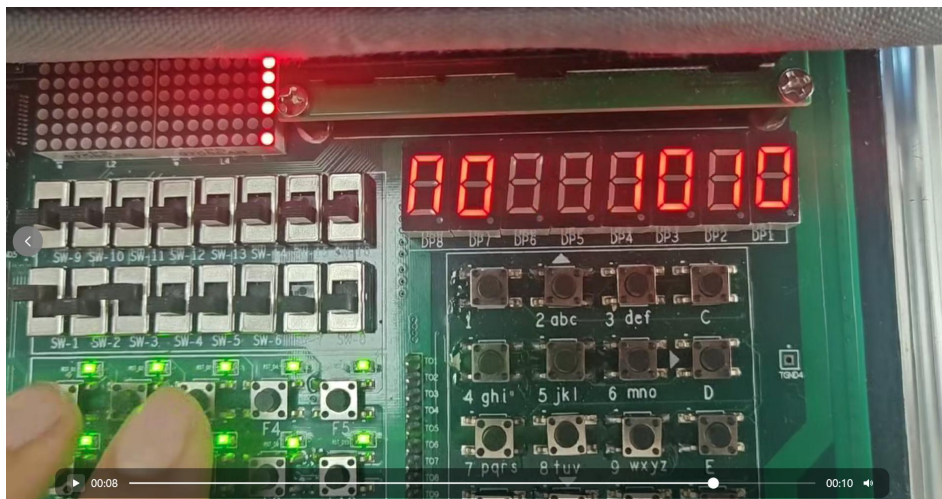
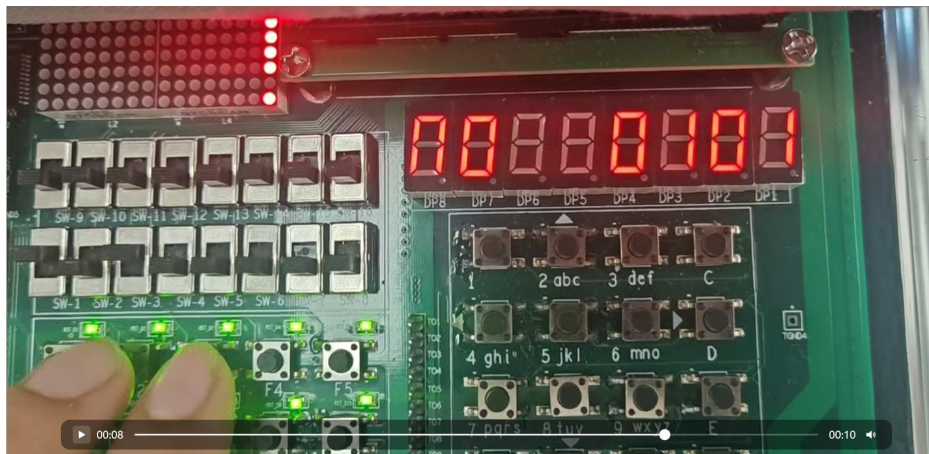
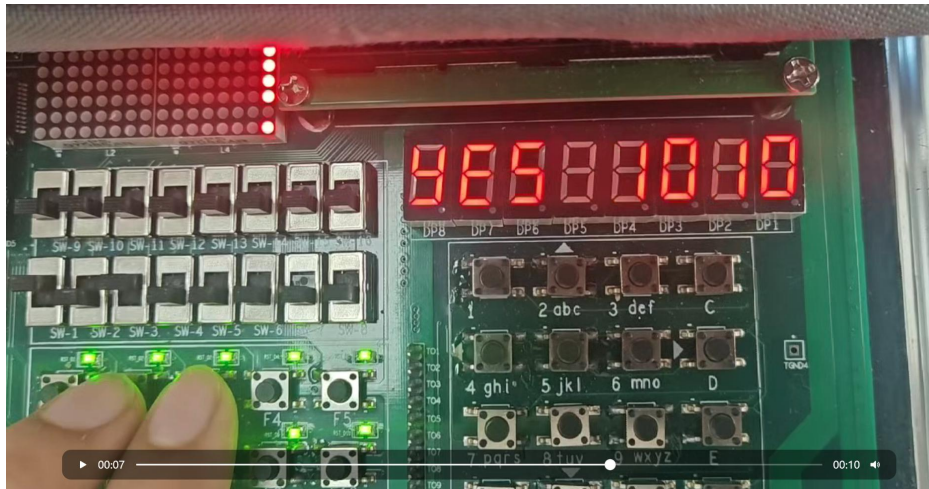
`next_state = {current_state[2:0], 1'b1}`;和 `next_state = {current_state[2:0], 1'b0}`;一定要放在检测到输入 1 或 0 的一开始，如果放在条件内，可能会被跳过。

这里输出 `match` 不会一直是 1，条件不成立自然为 0。

显示这里用的是 `next_state`，如果是 `current` 的话会过一个时钟周期才变。

防抖模块+驱动模块+管脚绑定情况和前文一致。

最后结果：



六、实验仿真波形

1. 重叠检测

编写 SequenceDetector_tb.v 代码

```
1  `timescale 1ns/1ns
2
3  module SequenceDetector_tb;
4      // -----
5      // 时钟产生 (10ns 周期 = 100MHz)
6      // -----
7      reg clk_sys = 0;
8      always #5 clk_sys = ~clk_sys;
9
10     // -----
11     // DUT 输入信号
12     // -----
13     reg resetn_b = 1;
14     reg key1_b = 1;
15     reg key0_b = 1;
16     reg [3:0] sw = 4'b0000;
17
18     // -----
19     // DUT 输出信号
20     // -----
21     wire [7:0] seg;
22     wire [7:0] sel; //
23
24     SequenceDetector #(5) uut (
25         .clk_sys(clk_sys),
26         .resetn_b(resetn_b),
27         .sw(sw),
28         .key1_b(key1_b),
29         .key0_b(key0_b),
30         .seg(seg),
31         .sel(sel)
32     );
33     //如果不判断是否重叠就用这个
34
35     // -----
36     // 仿真波形输出
37     // -----
38     initial begin
39         $dumpfile("SequenceDetector.vcd");
40         $dumpvars(0, SequenceDetector_tb);
41     end
42
43
44     initial begin
45         $display("=== SequenceDetector Testbench Start ===");
46
47         // 初始状态
48         resetn_b = 1;
49         key1_b = 1;
50         key0_b = 1;
51         sw = 4'b0000;
52         #50;
53
54         // 复位
55         resetn_b = 0;
56         #50;
57         resetn_b = 1;
58         #50;
59
60         // 设置目标 pattern = 1011
61         sw = 4'b1011;
62         #50;
63
64         // 模拟输入序列: 1,0,1,1
65         // 每个输入间隔 100ns
66         key1_b = 0; #10; key1_b = 1; #500; // 输入1
67         key0_b = 0; #10; key0_b = 1; #500; // 输入0
68         key1_b = 0; #10; key1_b = 1; #500; // 输入1
69         key1_b = 0; #10; key1_b = 1; #500; // 输入1 => 匹配应成立
70
71         // 等待一段时间观察非重叠行为
72         #300;
73
74         // 再输入一个新序列验证“非重叠”
75         key0_b = 0; #10; key0_b = 1; #500;
76         key1_b = 0; #10; key1_b = 1; #500;
77         key1_b = 0; #10; key1_b = 1; #500;
78         key1_b = 0; #10; key1_b = 1; #500;
79
80         #500;
81         $display("=== Simulation Finished ===");
82         $finish;
83     end
84 end
```

1) 编译文件

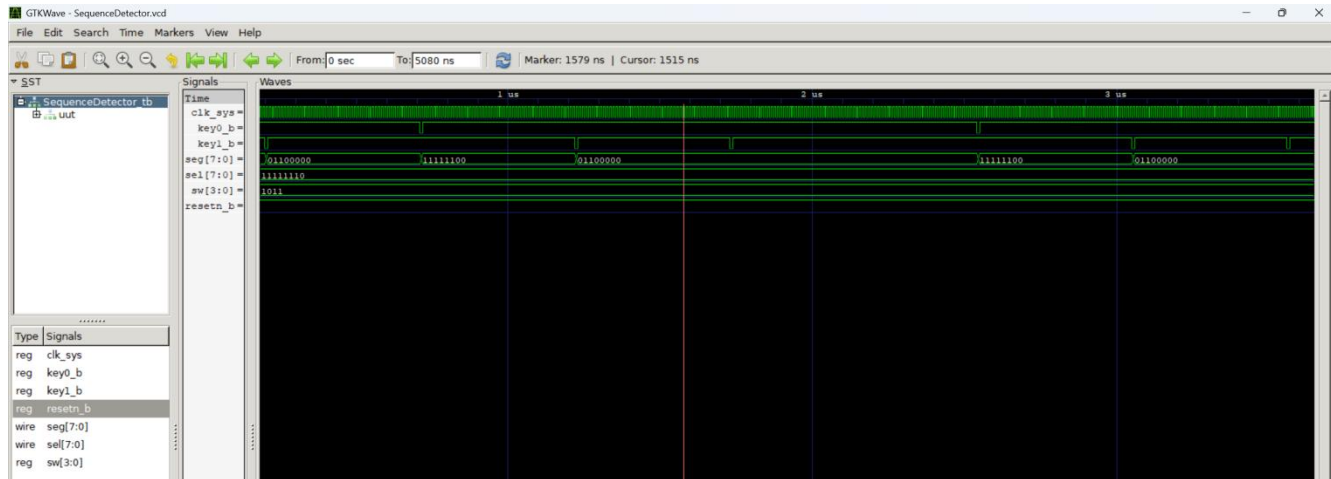
命令行: iverilog -o SequenceDetector.out SequenceDetector.v SequenceDetector_tb.v
debouncing.v scanDisplayDriver.v

2) 生成 .vcd 波形文件

命令行: vvp SequenceDetector.out

3) 观察波形

命令行: gtkwave SequenceDetector.vcd



2. 非重叠检测

编写 SequenceDetector_tb.v 代码（一模一样，只在实例化处有所不同）

```
// DUT 实例化
SequenceDetector_NonOverlap #(.delayT(5)) uut (
    .clk_sys(clk_sys),
    .resetn_b(resetn_b),
    .sw(sw),
    .key1_b(key1_b),
    .key0_b(key0_b),
    .seg(seg),
    .sel(sel)
);
//需要判断重叠的不算就用这个
initial begin
    $dumpfile("SequenceDetector_NonOverlap.vcd");
    $dumpvars(0, SequenceDetector_tb);
end

// SequenceDetector #(.delayT(5)) uut (
//     .clk_sys(clk_sys),
//     .resetn_b(resetn_b),
//     .sw(sw),
//     .key1_b(key1_b),
//     .key0_b(key0_b),
//     .seg(seg),
//     .sel(sel)
// );
// //如果不判断是否重叠就用这个

// // -----
// // 仿真波形输出
// // -----
// initial begin
//     $dumpfile("SequenceDetector.vcd");
//     $dumpvars(0, SequenceDetector_tb);
// end
```

1) 编译文件

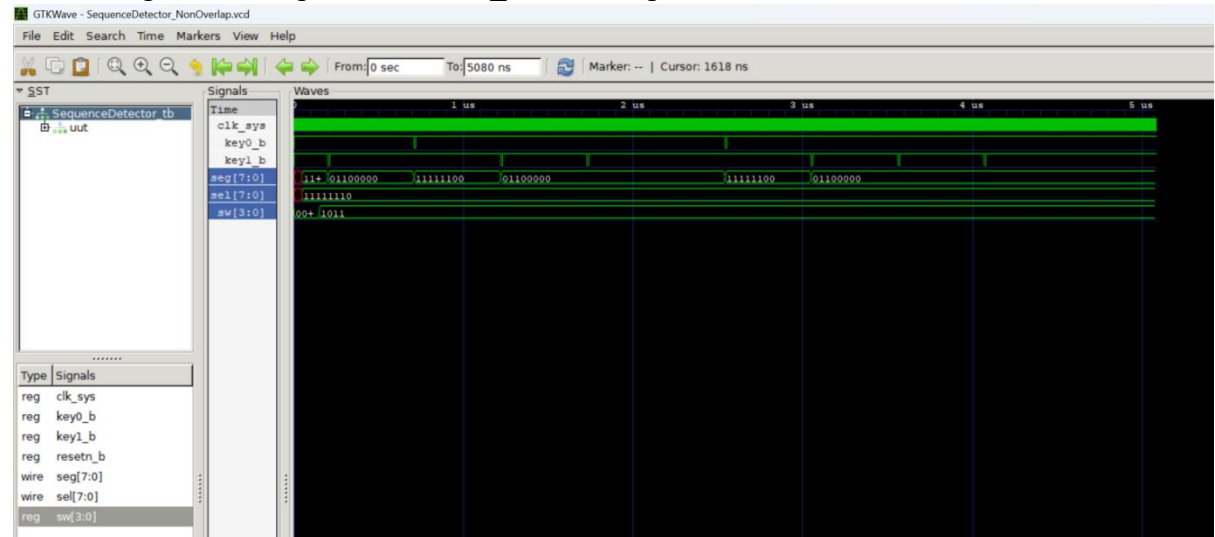
命令行: iverilog -o SequenceDetector.out SequenceDetector_NonOverlap.v
SequenceDetector_tb.v debouncing.v scanDisplayDriver.v

2) 生成 .vcd 波形文件

命令行: vvp SequenceDetector_NonOverlap.out

3) 观察波形

命令行: gtkwave SequenceDetector_NonOverlap.vcd



3. (额外) debouncing 防抖模块

编写 debouncing_tb.v 代码

```
`timescale 1ns/1ps

module debouncing_tb;

    // -----
    // 时钟产生 (周期10ns = 100MHz)
    // -----
    reg clk_sys = 0;
    always #5 clk_sys = ~clk_sys;

    // -----
    // DUT 输入 / 输出
    // -----
    reg key_b;
    reg resetn;
    wire key;
    wire key_pulse;
    wire [7:0]counter;

    // -----
    // DUT 实例化 (delayT很小方便观察)
    // -----
    debouncing_fsm #(
        .bitwidth(8),
        .delayT(50)//这里修改了位宽和delayT
    ) uut (
        .clk_sys(clk_sys),
        .resetn(resetn),
        .key_b(key_b),
        .key(key),
        .key_pulse(key_pulse),
        .counter(counter)
    );

    // 波形输出
    initial begin
        $dumpfile("debouncing.vcd");
        $dumpvars(0, debouncing_tb);
    end
end
```



```

// 测试流程
initial begin
    $display("=== Debouncing Testbench Start ===");
    // 初始状态
    resetn = 1;
    #50;
    // 复位
    resetn = 0;
    #30;
    resetn = 1;
    #50;

    key_b = 1; // 初始未按下
    #50;

    // 模拟有抖动的按键按下
    // 抖动序列: 1->0->1->0->1->0->保持0
    key_b = 0; #2;
    key_b = 1; #2;
    key_b = 0; #2;
    key_b = 1; #2;
    key_b = 0; #100; // 稳定按下 (低电平)

    // 保持按下状态一段时间
    #100;

    // 模拟抖动释放: 0->1->0->1->保持1
    key_b = 1; #2;
    key_b = 0; #2;
    key_b = 1; #2;
    key_b = 0; #2;
    key_b = 1; #100;

    // 等待波形稳定
    #200;

    $display("=== Simulation Finished ===");
    $finish;
end

```

1) 编译文件

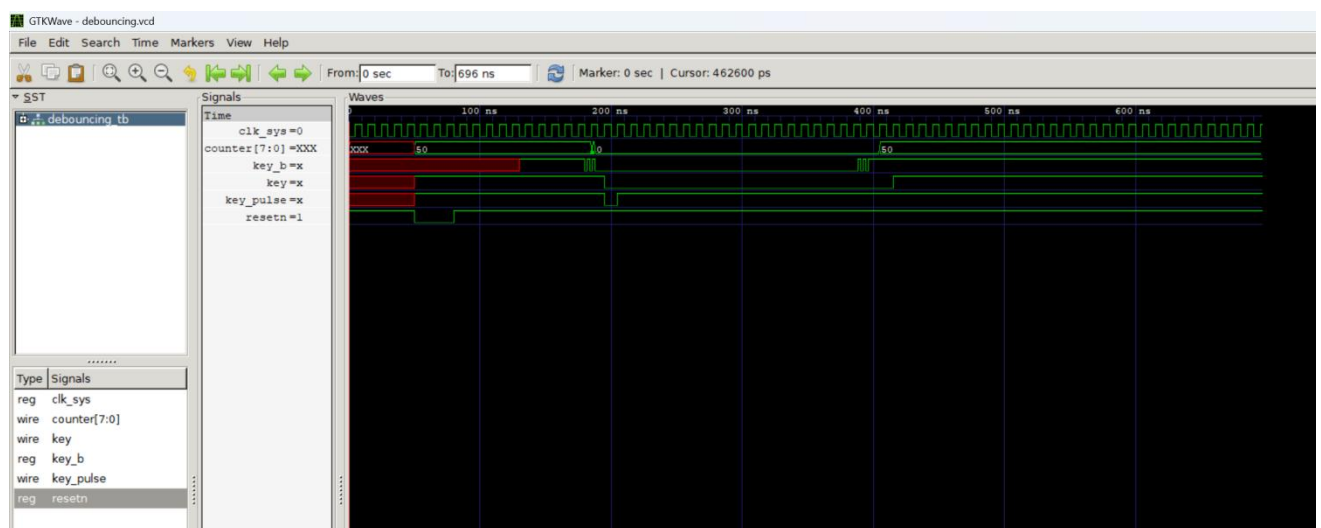
命令行: iverilog -o debouncing.out debouncing.v debouncing_tb.v

2) 生成 .vcd 波形文件

命令行: vvp debouncing.out

3) 观察波形

命令行: gtkwave debouncing.vcd



七、附加实验仿真波形

无附加实验，两个实验均为必做实验。